

PROJECT REPORT

Placement Portal Application

A Role-Based Full-Stack Web Application for Campus Recruitment

Kshitiz Kumar Singh | Roll: 25f1001872 | 25f1001872@ds.study.iitm.ac.in

IIT Madras Online Degree Programme | Data Science & Applications | 2026

1 AUTHOR

Full Name	Kshitiz Kumar Singh
Roll Number	25f1001872
Email	25f1001872@ds.study.iitm.ac.in
Programme	B.S. in Data Science & Applications — IIT Madras Online Degree
About	Currently pursuing Civil Engineering alongside the Data Science & Applications programme from IIT Madras. Primary interests include finance, algorithmic trading, and quantitative investing. Actively developing analytical and problem-solving skills to build a career in data-driven financial domains.

2 AI / LLM USAGE

Tool(s) Used	ChatGPT (OpenAI GPT-4)
Extent of Use	12–20% of total development effort
Areas Assisted	Flask route boilerplate, debugging, project structure, HTML structure, CSS Styling Ideas, and generating inline documentation/comments.
Code Ownership	All core logic — authentication flow, role-based access, schema design, and business constraints — was designed and implemented independently. AI was used only as a productivity aid, not as a primary author.

3 PROJECT DESCRIPTION

The Placement Portal is a role-based, full-stack web application built to modernize and centralize the campus recruitment process. It replaces manual spreadsheet-and-email coordination with a structured digital workflow that serves three distinct user roles: Admin, Company, and Student.

The Admin acts as gatekeeper — approving company registrations, overseeing all placement drives, managing users, and maintaining system integrity through blacklisting. Companies register, build profiles (pending Admin approval), post placement drives, and manage the applicant pipeline. Students browse approved drives, apply, upload resumes, and track their application status in real time.

The system enforces strict constraints: a student cannot apply to the same drive twice (UniqueConstraint at DB level), only approved companies can post drives, and blacklisted accounts lose access while preserving data integrity. The entire database is created programmatically via `db.create_all()` on first run with the Admin account auto-seeded — no manual setup required.

4 TECHNOLOGIES USED

Technology / Library	Version	Purpose & Reason for Use
Python	3.10+	Primary backend language; chosen for readability and strong Flask ecosystem support.
Flask	3.x	Lightweight WSGI micro-framework; provides routing, Blueprint organisation, and request handling.
Flask-SQLAlchemy	Latest	ORM layer over SQLite; allows Pythonic model definitions and eliminates raw SQL.
SQLite	Built-in	Zero-configuration embedded relational database; ideal for development and academic projects.
Werkzeug Security	Bundled	Password hashing via generate_password_hash / check_password_hash — bcrypt-backed.
Flask-Login	Latest	Session management and user authentication; provides login_required decorator pattern.
Jinja2	Bundled	Server-side HTML templating engine; enables dynamic page rendering with template inheritance.
Bootstrap 5	5.x	Responsive CSS framework; ensures consistent UI across devices with minimal custom CSS.
HTML5 / CSS3 / JS	—	Frontend markup, styling.

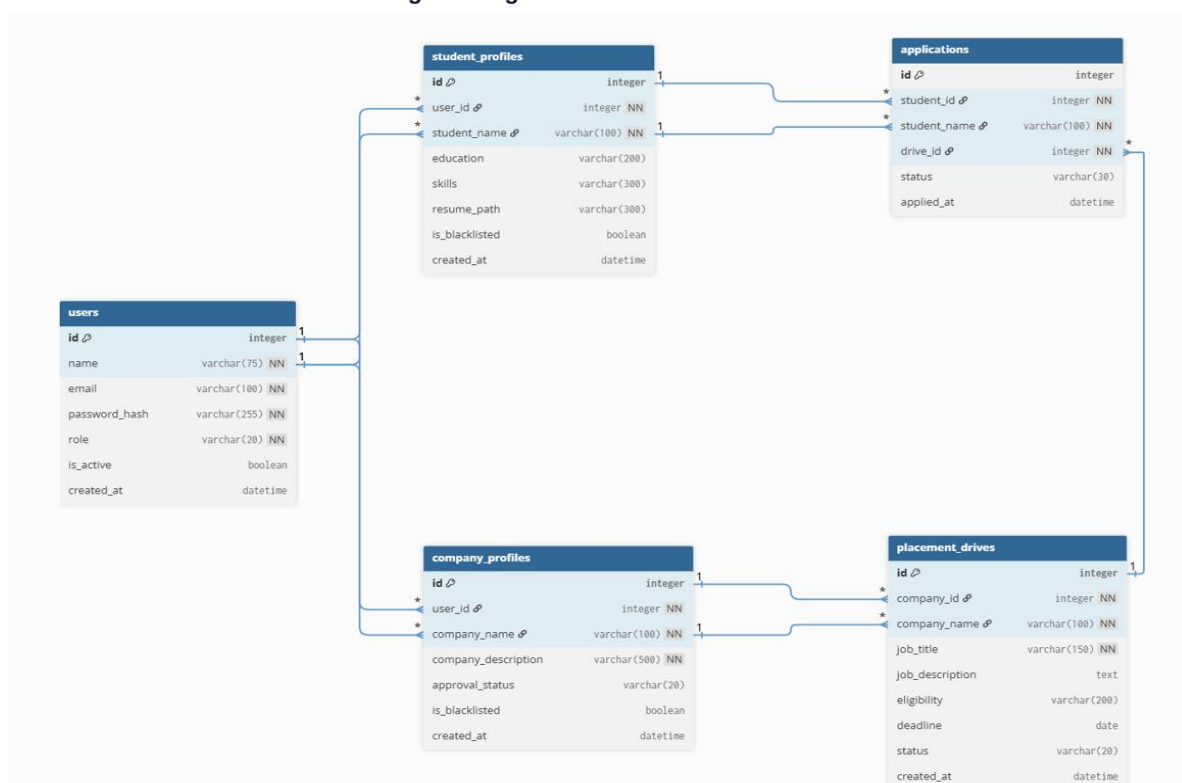
5 DB SCHEMA DESIGN

The schema follows third-normal-form normalisation, separating concerns across five distinct tables. The central users table acts as the shared authentication anchor for all three roles, with one-to-one satellite tables (student_profiles, company_profiles) hanging off it via foreign keys. Placement activity is captured in placement_drives (owned by companies) and applications (linking students to drives). A UniqueConstraint on (student_id, drive_id) in the applications table enforces the no-duplicate-application rule at the database level, independent of application-layer checks.

Table	Column	Type	Constraints / Notes
users	id	INTEGER	PK, Auto-increment
	name	VARCHAR(75)	NOT NULL
	email	VARCHAR(100)	NOT NULL, UNIQUE
	password_hash	VARCHAR(255)	NOT NULL — bcrypt hash
	role	VARCHAR(20)	NOT NULL — 'admin' 'company' 'student'
	is_active	BOOLEAN	DEFAULT True
student_profiles	created_at	DATETIME	DEFAULT utcnow
	id	INTEGER	PK, Auto-increment
	user_id	INTEGER	FK → users.id, UNIQUE, NOT NULL
	student_name	VARCHAR(100)	FK → users.name, NOT NULL
	education	VARCHAR(200)	Nullable
	skills	VARCHAR(300)	Nullable
	resume_path	VARCHAR(300)	Path to uploaded PDF resume
company_profiles	is_blacklisted	BOOLEAN	DEFAULT False — set by Admin
	created_at	DATETIME	DEFAULT utcnow
	id	INTEGER	PK, Auto-increment
	user_id	INTEGER	FK → users.id, UNIQUE, NOT NULL
	company_name	VARCHAR(100)	FK → users.name, NOT NULL

	company_description	VARCHAR(500)	NOT NULL
	approval_status	VARCHAR(20)	DEFAULT 'Pending' — 'Pending' 'Approved'
	is_blacklisted	BOOLEAN	DEFAULT False — set by Admin
	created_at	DATETIME	DEFAULT utcnow
placement_drives	id	INTEGER	PK, Auto-increment
	company_id	INTEGER	FK → company_profiles.id, NOT NULL
	company_name	VARCHAR(100)	FK → company_profiles.company_name, NOT NULL
	job_title	VARCHAR(150)	NOT NULL
	job_description	TEXT	Nullable
	eligibility	VARCHAR(200)	Nullable
	deadline	DATE	Application closing date
	status	VARCHAR(20)	DEFAULT 'Active' — 'Active' 'Closed'
	created_at	DATETIME	DEFAULT utcnow
applications	id	INTEGER	PK, Auto-increment
	student_id	INTEGER	FK → student_profiles.id, NOT NULL
	student_name	VARCHAR(100)	FK → student_profiles.student_name, NOT NULL
	drive_id	INTEGER	FK → placement_drives.id, NOT NULL
	status	VARCHAR(30)	DEFAULT 'Waiting' — Waiting Shortlisted Rejected
	applied_at	DATETIME	DEFAULT utcnow
	UNIQUE CONSTRAINT	—	UNIQUE(student_id, drive_id) — prevents duplicate applications

ER Diagram — generated from schema models:



6 API DESIGN

The application uses Flask's Blueprint system to organise routes into four logical modules: auth, admin, company, and student. All routes are server-rendered (Jinja2 templates) rather than REST JSON endpoints — form submissions use POST with Flask redirects following the Post/Redirect/Get pattern. The complete YAML specification is submitted as a separate file.

Access control is enforced at the route level: company routes check `is_blacklisted` and company ownership (`drive.company_id` vs session user); student routes check `is_blacklisted` before rendering any content; admin routes are accessible only to the seeded admin session. Unauthenticated or unauthorized access returns 403 or renders an error string.

Blueprint	Endpoint	Method	Description
auth	/	GET	Landing / home page
	/login	GET, POST	Unified login with role-based redirect to Admin / Company / Student dashboard
	/logout	GET	Clears Flask session and redirects to home
	/register/student	GET, POST	Student registration — creates User + StudentProfile, handles resume upload
	/register/company	GET, POST	Company registration — creates User + CompanyProfile with Pending approval status
admin	/admin/dashboard	GET	Aggregated stats: total students, companies, pending approvals; lists all entities
	/admin/search	GET	Search user by ID; redirects to appropriate company or student profile
	/admin/approve_company/<id>	POST	Sets company approval_status to Approved
	/admin/blacklist_company/<id>	POST	Sets company is_blacklisted to True
	/admin/blacklist_student/<id>	POST	Sets student is_blacklisted to True
	/admin/company_profile/<id>	GET	View full company profile details
	/admin/student_profile/<id>	GET	View student profile and their application list
	/admin/drive_details/<id>	GET	View individual placement drive details
	/admin/drive_status/<drive_id>	POST	Force-closes a placement drive (status → Closed)
	/admin/student_applications/<id>	GET	View all applications submitted by a specific student
company	/company/dashboard	GET	Company overview: active and closed drives with stats; blacklist check
	/company/create_drive	GET, POST	Create a new placement drive (job_title, description, eligibility, deadline)
	/company/drive_applications/<drive_id>	GET	List all student applications for a specific drive
	/company/student_application/<id>	GET	View individual student application details and resume info

	/company/student_application/<id>/update_status	POST	Update application status: Waiting Shortlisted Rejected
	/company/drive_status/<drive_id>	POST	Close an active placement drive
student	/student/dashboard	GET	Student home: approved companies, applied drives; blacklist check
	/student/company_profiles/<company_id>	GET	View a company's profile and their active/closed drives
	/student/drive_details/<drive_id>	GET	View full details of a single placement drive
	/student/drive_details/<drive_id>/apply	POST	Apply to a drive; duplicate check via DB UniqueConstraint
	/student/applications_history	GET	View full application history with real-time status tracking

7 ARCHITECTURE & FEATURES

Project Organisation

The project uses the Flask Application Factory pattern. `create_app()` in `app/__init__.py` initialises all extensions (SQLAlchemy), creates the DB schema, seeds the Admin, and registers all Blueprints. This keeps the app object decoupled from global state and makes testing straightforward. Each Blueprint (`auth_bp`, `admin_bp`, `company_bp`, `student_bp`) is a self-contained module with its own routes file — co-located logic for each role. Models are in `app/models/`, one file per entity. Templates follow the same role-based hierarchy under `app/templates/` (`admin/`, `company/`, `student/` subdirectories). Static assets including uploaded resumes live under `app/static/`.

Path	Contents & Role
<code>run.py</code>	Application entry point — invokes <code>create_app()</code> factory and starts the dev server.
<code>app/__init__.py</code>	App factory — initialises Flask, SQLAlchemy, creates DB via <code>db.create_all()</code> , seeds the Admin account on first run, and registers all four Blueprints.
<code>app/config.py</code>	Centralised configuration: <code>SECRET_KEY</code> , SQLite DB URI constructed with <code>os.path.abspath</code> for portability.
<code>app/models/</code>	One file per model: <code>user.py</code> , <code>student.py</code> , <code>company.py</code> , <code>placement_drive.py</code> , <code>application.py</code> — each defines SQLAlchemy columns, FKs, and relationships.
<code>app/routes/</code>	Four Blueprint modules: <code>auth_routes.py</code> , <code>admin_routes.py</code> , <code>company_routes.py</code> , <code>student_routes.py</code> — each handles one role's full request lifecycle.
<code>app/templates/</code>	Jinja2 HTML templates organised by role: <code>admin/</code> , <code>company/</code> , <code>student/</code> sub-folders plus shared <code>login.html</code> , <code>home.html</code> , and registration pages.
<code>app/static/</code>	CSS stylesheets, JavaScript files, and <code>uploads/resumes/</code> directory where student PDFs are stored using a deterministic naming scheme.
<code>instance/placement.db</code>	Auto-generated SQLite database file — created on first run, never committed to version control.

Features Implemented

Role / Module	Feature	Implementation Detail
Authentication	Unified Login + Role Redirect	Single <code>/login</code> endpoint; role field in session determines redirect target (Admin/Company/Student dashboard).
	Secure Password Hashing	<code>Werkzeug generate_password_hash / check_password_hash</code> — <code>bcrypt</code> under the hood; plaintext never stored.
	Session Management	Flask session stores <code>user_id</code> , email, role; cleared on logout.
	Admin Auto-Seed	App factory checks for existing admin on startup; creates default credentials if absent — no public registration.

Admin	System Dashboard	Aggregates total students, companies, pending approvals; lists all entities with inline action buttons.
	Company Approval Workflow	New companies default to Pending; Admin must Approve before login is granted — enforced in login route.
	Blacklisting (Student/Company)	Sets is_blacklisted flag; blocked at dashboard entry — data preserved, access revoked.
	User Search by ID	GET /admin/search?id=X queries User table, resolves role, and redirects to the correct profile view.
	Drive Force-Close	Admin can close any active drive regardless of company action.
Company	Drive Management (Full CRUD)	Create drives with title, description, eligibility, and deadline; close drives via status toggle.
	Applicant Pipeline Management	View all applicants per drive; update each status (Waiting → Shortlisted / Rejected) via POST form.
	Authorisation Check	Company can only view drives/applications belonging to their own company_id — 403 returned otherwise.
Student	Browse Approved Drives	Dashboard shows only Approved, non-blacklisted companies; filters ensure students see valid opportunities.
	Apply to Drive	POST /apply checks for existing Application record; duplicate prevented at DB level via UniqueConstraint.
	Resume Upload	PDF saved to static/uploads/resumes/ with deterministic filename student_{id}_resume.pdf.
	Application History & Status	Real-time status tracking (Waiting / Shortlisted / Rejected) rendered in applications_history.html.
System	Duplicate Application Prevention	Database-level UniqueConstraint on (student_id, drive_id) in applications table — enforced even if flash check is bypassed.
	Approval Gate	Companies with Pending status cannot access dashboard post-login — checked in both login route and dashboard.
	Auto DB Creation	db.create_all() inside app factory; no migration commands or manual schema setup required.

8 VIDEO SUBMISSION

The video walkthrough demonstrates all three user role flows — Admin approval workflow, Company drive creation and applicant management, and Student registration, application, and status tracking.

[▶ VIDEO DEMO LINK](#)